Git Workflow

This is the branching workflow we'll use for the project. It's based on a standard, widely-used team workflow (a simplified GitFlow), adapted for our team size.

The three branches

`main`

- Always stable, always working. This is what actually gets published/deployed.
- Never edited directly and never pushed to directly, except for changes that don't affect code (e.g. fixing a typo in a README).
- Any code change reaches `main` only through a Pull Request (PR) from `dev`.

`dev`

- The "everything in progress" branch. Less critical than `main`, but still treated carefully.
- Receives PRs from feature branches.
- Once code on `dev` is fully tested and confirmed working, it gets PR'd into `main`.

Feature branches

- Created from `dev` whenever starting a new piece of work (a feature, a fix, etc.).
- Named clearly enough that anyone can tell what it's for (e.g. `feature/contact-form`, `fix/broken-nav-link`).

- Deleted once merged — they're temporary by design.

How merging works (and why)

Merge direction	Method	Why
feature → `dev`	**Squash and merge**	Combines all the small commits from the feature branch into one clean commit on `dev`. Fine to do since the feature branch is deleted right after anyway.
`dev` → `main`	**Regular merge (no squash)**	Squashing rewrites commit history. If `main` and `dev` end up with different versions of the same history (squashed vs. not), Git can see them as different changes even when the content is identical — causing avoidable conflicts later. A regular merge keeps history consistent between the two branches.

Daily habits

- **Before starting any work:** `fetch` and `pull` to make sure you're working on the latest version of the branch.
- **While working:** commit often, in small logical chunks, with clear messages.
- **Push regularly** — don't wait until a feature is fully done. Committing alone only saves your work locally; pushing is what actually protects it if something happens to your machine.
- Keep feature branches short-lived. Small, frequent PRs are easier to review and less likely to run into conflicts than one large branch open

for weeks.

Force-push policy

- `push --force` is **never** used on `main` or `dev`, under any normal circumstance.
- The only exception is a genuine recovery situation, and only after every team member is aware and agrees — never as a routine fix.

Recommended (if available to us)

- Turn on branch protection rules on `main` (and ideally `dev`) in GitHub/GitLab — this blocks direct pushes and enforces PRs automatically, so the rules don't rely purely on remembering to follow them.